Multi-Vendor Bookstore System

Architecture & Smart Recommendation Specification (Bilingual Guide)

Doc Version: 2.0.0
Status: Approved Architecture

Executive Summary & Core Concept (මූලික සංකල්පය)

This platform establishes a decentralized, multi-vendor marketplace connecting distributed bookshops into a unified network. When a customer queries a book that is out of stock (Stock = 0) at their selected bookstore, the platform automatically detects inventory across neighboring network partner shops and facilitates seamless routing.
(එක් bookshop එකක පොතක් නැති වූ විට customer ට වෙනත් පාශ්වකාර bookshop එකකින් ලබාගැනීමට විකල්ප සපයන ස්වයංක්‍රිය පද්ධතියකි.)

01 Core Value Proposition (ප්‍රධාන වාසි)

For Readers / Buyers

No need to physically visit multiple shops. Real-time comparison of stock availability, prices, distance, and express delivery options.

For Bookstore Vendors

Small independent shops gain immediate digital reach. Inter-store order routing maintains customer loyalty without missed sales.

AI Personalization

Real-time content-based recommendation engine suggesting relevant titles based on past purchasing vectors and author affinities.

02 Inventory Routing & Out-of-Stock Workflow

The platform enforces a decoupled inventory aggregation pattern. When a customer initiates a search, the routing algorithm executes the following sequence:

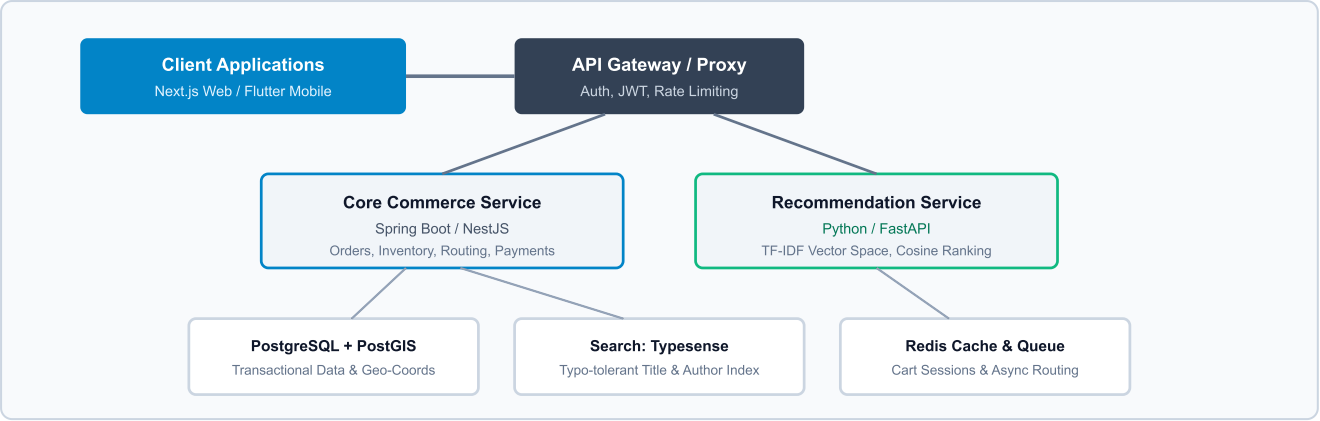
1. **Global Catalog Query:** The search query resolves against the Master Books Catalog using unified ISBN or metadata index.
2. **Local Stock Evaluation:** System queries the chosen vendor's local stock. If `stock_quantity > 0`, regular checkout is served.
3. **Out-of-Stock Detection & Auto-Routing:** If local stock is 0, the system triggers the *Partner Routing Engine*.
4. **Proximity & Cost Ranking:** Partner vendors holding the requested title are filtered and ranked using three weighted parameters:
   - **Distance (Geodesic):** Nearest physical shop coordinates via PostGIS.
   - **Total Landed Cost:** Book unit price + calculated courier dispatch rate.
   - **Vendor SLA:** Historical dispatch time rating and fulfillment success rate.
5. **Customer Choice / Backorder:** The customer can either accept the best recommended partner with 1-click or let the system auto-transfer the order.

Fail-Safe Mechanism (ගනුදෙනු ආරක්ෂණ ක්‍රමවේදය)

If an assigned secondary vendor fails to accept the routed order within 60 minutes, the order is automatically cascaded to the next best partner shop in the network, or a full refund/store credit is triggered automatically.

03 System Architecture & Service Boundaries

The system utilizes a modular, microservices-ready decoupled pattern. Core transaction flows are separated from the compute-intensive recommendation routines.



04 Recommended Technology Stack

Layer / Role	Selected Technology	Design Justification
Storefront & Portal	Next.js (React) + Tailwind CSS	Server-Side Rendering (SSR) ensures book titles are indexed by search engines. Highly responsive UI.
Vendor Mobile Apps	Flutter (Dart)	Unified codebase compiling to Android and iOS for vendor order fulfillment and customer mobile browsing.
Core Backend	Spring Boot (Java) / NestJS	Robust enterprise transaction boundaries, strong typing, clean routing of complex order workflows.
AI Recommendation	Python (FastAPI + Scikit-learn)	Lightweight microservice optimized for vector calculations, matrix operations, and high-concurrency read queries.
Geospatial Database	PostgreSQL + PostGIS	Allows native spatial SQL queries: `ST_DWithin` to find bookstores within radial proximity of the buyer.
Logistics Integrations	Local Couriers (Domex/PromptX)	Automated API consignment dispatch, tracking waybill creation, and status webhooks.

05 Relational Data Schema Design

The schema separates universal book definitions from individual bookstore holdings to allow multiple sellers per title.

Table 1: Master Books (books)

Column	Type	Constraints	Description & Purpose
id	UUID	PRIMARY KEY	Unique global book record identifier
isbn	VARCHAR(20)	UNIQUE, NOT NULL	Standard ISBN-10 / ISBN-13 identifier
title	VARCHAR(255)	INDEXED	Official publication title
author	VARCHAR(255)	INDEXED	Primary author / editor credentials
category	VARCHAR(100)	NOT NULL	Genre (e.g. Science Fiction, Academic, History)
tags	TEXT[]	NULLABLE	Vector input keywords (e.g. ['thriller', 'space'])

Table 2: Store Inventory Link (store_inventory)

Column	Type	Constraints	Description & Purpose
id	UUID	PRIMARY KEY	Unique stock line ID
store_id	UUID	FK: bookstores	References physical store holding the item
book_id	UUID	FK: books	References universal master book record
price	NUMERIC(10,2)	NOT NULL, > 0	Specific store's selling price (LKR / USD)
stock_quantity	INTEGER	NOT NULL, ≥ 0	Available physical copies in warehouse
condition	VARCHAR(20)	DEFAULT 'NEW'	Book condition: NEW, USED_EXCELLENT, USED_GOOD

Table 3: Bookstores Physical Data (bookstores)

Column	Type	Constraints	Description & Purpose
id	UUID	PRIMARY KEY	Unique partner merchant ID
name	VARCHAR(255)	NOT NULL	Registered commercial store name
location	GEOGRAPHY(Point)	SRID 4326	Latitude & Longitude for radial geospatial search
is_active	BOOLEAN	DEFAULT TRUE	Store status (toggled during holidays/closures)

06 Recommendation Engine Architecture

To eliminate the Cold-Start dilemma common in new platforms, the system implements **Content-Based Vector Space Modeling** using TF-IDF and Cosine Proximity rather than purely Collaborative Filtering.

Vector Similarity Formulation:

The customer preference vector ($\mathbf{u}$) is calculated from the mean embeddings of their past purchases. Similarity with candidate catalog item ($\mathbf{b}$) is formulated as:

$$\text{Similarity}(\mathbf{u}, \mathbf{b}) = \cos(\theta) = (\mathbf{u} \bullet \mathbf{b}) / (||\mathbf{u}|| \times ||\mathbf{b}||) = [\sum u_i b_i] / [\sqrt{(\sum u_i^2)} \times \sqrt{(\sum b_i^2)}]$$

FastAPI Recommendation Microservice Implementation

```
import numpy as np import pandas as pd from sklearn.feature_extraction.text import TfidfVectorizer from sklearn.metrics.pairwise import cosine_similarity from fastapi import FastAPI, HTTPException from pydantic import BaseModel from typing import List app = FastAPI(title="Book Recommendation Engine") # Metadata combination: Author + Category + Tags books_df["features"] = books_df["author"] + " " + books_df["category"] + " " + books_df["tags"] tfidf = TfidfVectorizer(stop_words="english") tfidf_matrix = tfidf.fit_transform(books_df["features"]) class RecRequest(BaseModel): purchased_ids: List[str] top_n: int = 4 @app.post("/api/v1/recommend") def get_recommendations(payload: RecRequest): indices = books_df[books_df["id"].isin(payload.purchased_ids)].index.tolist() if not indices: raise HTTPException(status_code=404, detail="No history found") # Calculate Mean Preference Vector user_vector = np.asarray(tfidf_matrix[indices].mean(axis=0)) scores = cosine_similarity(user_vector, tfidf_matrix).flatten() ranked = scores.argsort()[::-1] recommendations = [] for idx in ranked: b_id = books_df.iloc[idx]["id"] if b_id not in payload.purchased_ids: recommendations.append({ "book_id": b_id, "title": books_df.iloc[idx]["title"], "score": round(float(scores[idx]), 3) }) if len(recommendations) >= payload.top_n: break return {"results": recommendations}
```

07 3-Phase Delivery Roadmap

Phase 1: MVP Core

Master catalog aggregation, 5 onboarded shops, store-level inventory management, and Cash-on-Delivery (COD) checkout.

Phase 2: Auto-Routing

PostGIS distance radius queries, automated out-of-stock alternative suggestion modals, and card payment gateway integration.

Phase 3: AI & Logistics

Production FastAPI recommendation engine deployment, personalized home feeds, and 3rd-party courier API auto-waybill generation.